A Compute-in-Memory Based on Approximate Multiplication with a Calibration Adder

Seunggu Choi, Minkyu Song, and Soo Youn Kim
Department of Semiconductor Science, Dongguk University, Seoul, 04620, Korea
Email : {hycs0403, mksong, sooyoun}@dgu.ac.kr

2026 IEEE International Symposium on Circuits and Systems (ISCAS) | 979-8-3315-7769-8/26/$31.00 ©2026 IEEE | DOI: 10.1109/ISCAS66217.2026.11562200

***Abstract*— This paper presents an energy-efficient compute-in-memory (CIM) architecture with approximated multiplication and a calibration adder. The proposed CIM applies Mitchell's approximate multiplication to implement multiply-accumulate (MAC) with a single addition operation for multiplication, regardless of input precision, thereby reducing both latency and power consumption of arithmetic operations. Furthermore, the precision degradation caused by approximate multiplication can be improved by approximately 3.4% on average using the proposed calibration adder (C-Adder). Each computing element utilizes an 8T-SRAM-based structure with separate local and global read bit lines to enable parallel MAC operations and further enhance computational throughput. Implemented in a 28-nm CMOS process, the proposed CIM achieves 38.65 TOPS/W energy efficiency, 0.05003 TOPS/mm² area efficiency, and 91.86 % inference accuracy on ResNet-20 with CIFAR-10.**

Keywords*— *Calibration Adder, Compute-In-Memory, Mitchell Approximate Multiplication

I. Introduction

The energy efficiency and performance of deep learning systems are primarily constrained by data movement in conventional von Neumann architectures. Frequent transfers between processing cores and memory lead to the well-known "memory bottleneck", which severely limits system scalability [1]. To overcome this issue, compute-in-memory (CIM) architectures have attracted significant attention as a promising solution [2]. CIM performs multiply-accumulate (MAC) operations directly within or near memory arrays, thereby minimizing data movement and improving energy efficiency (TOPS/W) [3].

Recently, digital CIM architectures have gained significant attention as an alternative to analog CIM, which faces challenges such as analog-to-digital converter and digital-to-analog converter overhead and variation sensitivity. Digital CIM eliminates such issues by operating fully in the digital domain, providing deterministic accuracy, stable operation under process, voltage, and temperature (PVT) variations, and compatibility with standard-cell design flows.

This work was supported in part by National R&D Program through the National Research Foundation of Korea (NRF) funded by Ministry of Science and ICT(RS-2024-00402495), in part by the Technology Innovation Program (Public-private joint investment semiconductor R&D program(K-CHIPS) to foster high-quality human resources) (RS-2023-00235137, 50%) funded by the Ministry of Trade, Industry & Energy(MOTIE, Korea)(1415187379), in part by the Korea government(Ministry of Science and ICT)(IITP-2026-RS-2024-00438007), and in part by the BK21 FOUR (Fostering Outstanding Universities for Research) funded by the Ministry of Education (MOE, Korea) and National Research. Foundation of Korea (NRF).

The chip fabrication and EDA tool were supported by the IC Design Education Center (IDEC), Korea.

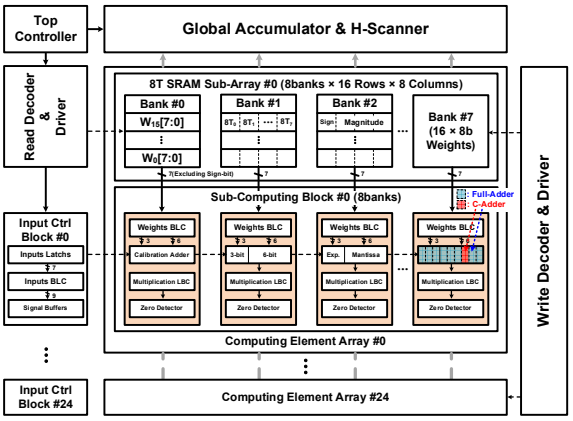


Fig. 1 Overall architecture of the proposed CIM.

Accordingly, recent studies have explored various circuit-level optimizations to improve the efficiency of digital CIM macros [4]. For example, several works have focused on enhancing area efficiency through SRAM-based logic integration [5] and improving energy efficiency using eDRAM-based digital in-memory computing techniques [6]. Nevertheless, because digital CIM performs arithmetic operations sequentially along accumulation paths, it suffers from limited parallelism and increased latency. Thus, while digital CIM offers high reliability and energy efficiency, structural improvements are required to enhance parallel operation.

To meet these requirements, we propose a digital CIM architecture using Mitchell approximate multiplication (MAM). The MAM approach replaces conventional multi-step multiplications with a single addition in the logarithmic domain, reducing both power consumption and latency. However, MAM incurs a loss of precision and tends to approximate values smaller than the actual values. Therefore, in this study, we reduced the approximation error caused by MAM using a calibration adder(C-adder). Furthermore, the C-adder compensates for underestimation without additional area overhead. Therefore, the proposed CIM based on approximate multiplication with C-adder achieves a balanced improvement in energy efficiency, precision, and latency through controllable circuit-level calibration, providing an effective solution for power-efficient digital CIM design.

II. Approximate Multiplication-Based CIM

The proposed CIM architecture comprises a top-level controller, write/read drivers, input control blocks, an 8T-SRAM–based computing element (CE) array, a global accumulator, and an H-Scanner. As shown in Fig. 1, each CE consists of an 8T-SRAM sub-array and a sub-computing block (SCB). Within each SCB, a Mitchell-based approximate multiplication unit (MAMU) performs the core logarithmic-domain computation. The stored weight data in the 8T-SRAM

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 30,2026 at 12:12:55 UTC from IEEE Xplore. Restrictions apply.

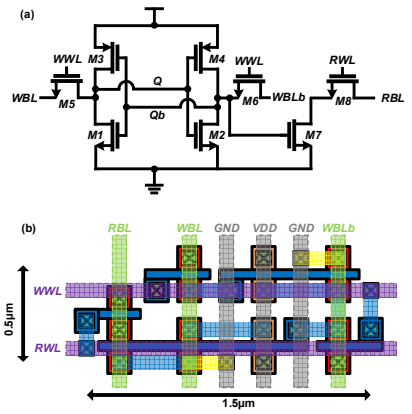


Fig. 2 (a) Schematic and (b) layout of the 8T- SRAM cell.

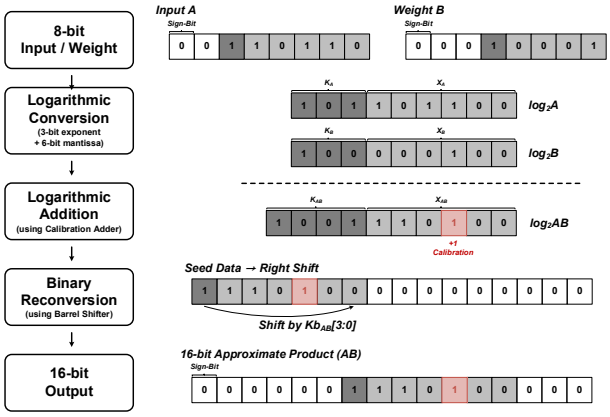


Fig. 3 Log-domain computation flow.

sub-array are first converted from the binary domain to the logarithmic domain through a binary-to-log converter (BLC), which forms the initial stage of the MAMU. The converted logarithmic weights are then combined with the logarithmic input data through a single addition operation using a C-adder. In this adder, only the 2^2-bit position is replaced by a calibration cell (highlighted in red in Fig. 1) to compensate for the dominant error component of the Mitchell approximation, while the remaining bit positions are implemented with conventional full adders (shown in blue). The resulting sum in the logarithmic domain is subsequently reconverted into the binary domain via a log-to-binary converter (LBC), producing an approximate multiplication result. Finally, a zero detector identifies inactive data paths to reduce unnecessary accumulation operations. The partial products generated by all CEs are accumulated by the global accumulator unit to obtain the final output, as illustrated in Fig. 1**.**

A. Compact 8T-SRAM Cell

In this work, an 8T SRAM cell is adopted to store the weight data. The schematic and layout of the implemented 8T cell are shown in Fig. 2 (a) and (b)**,** respectively**.** In conventional 6T-SRAM cells, read and write operations share the same bit lines (BL/BLB). As a result, during a read operation, voltage variations on the bit lines may disturb the stored data in the storage node, causing read–write interference. This issue becomes more critical in CIM arrays consisting of a large number of cells, where such interference can accumulate and degrade computational stability. In contrast, the 8T-SRAM cell adds two dedicated read transistors to form an independent read port (RBL/RWL), physically separating the read and write paths [5,7]. This separation ensures stable storage node voltage during read operations and eliminates interference. The two additional transistors also contribute to the local conversion (encoder-side) path, while the decoupled read port enables bank-level parallel access among different sub-arrays and improves read stability under PVT variations. The 8T-SRAM cell is implemented using a 28-nm CMOS process, and the fabricated bit-cell size is 1.5 μm × 0.5 μm.

B. Mitchell Approximate Multiplication

In conventional digital CIM architectures, multiplication is performed through repeated partial-product additions or Booth encoding, which increases circuit complexity, latency, and power consumption. To simplify multiplication, the MAM can replace conventional multiplication with a single addition in the logarithmic domain [8]. The MAM operation is as follows: a binary number N can be approximated as

$$N \approx 2^k(1+x) \tag{1}$$

where k denotes the position of the most significant '1' and x represents the fractional part of the remaining bits. The value x is linearly approximated within the range of [0, 1). Using this formulation, the logarithmic conversion becomes

$$\log_2 N \approx k + x \tag{2}$$

Therefore, the multiplication of two numbers N and M can be rewritten as a single addition in the log domain:

$$\log_2 NM = \log_2 N + \log_2 M$$

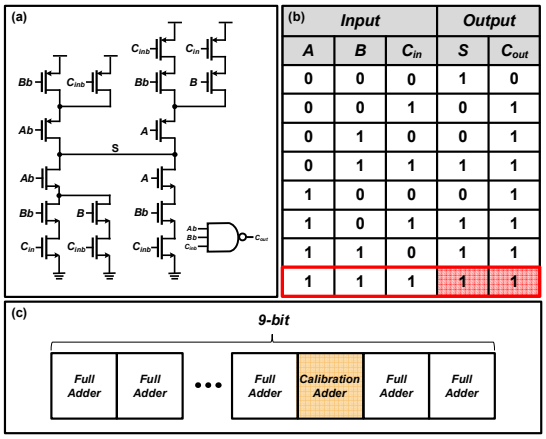
$$\approx (k_N + x_N) + (k_M + x_M) \tag{3}$$

This logarithmic transformation eliminates iterative partial-product accumulation, improving both latency and energy efficiency. Because Mitchell's approximation exhibits systematic underestimation, *this work* introduces a C-Adder to inject a small positive calibration in the log-domain addition with minimal overhead; the overall flow—from logarithmic conversion and calibrated addition to binary reconversion—is illustrated in Fig. 3.

Specifically, for $A = 2^{\varepsilon_A}(1+f_A)$ and $B = 2^{\varepsilon_B}(1+f_B)$, the ideal product is $AB = 2^{\varepsilon_A+\varepsilon_B}(1+f_A+f_B+f_Af_B)$, while Mitchell uses $\widehat{AB} = 2^{\varepsilon_A+\varepsilon_B}(1+f_A+f_B)$, leading to an underestimation by the omitted term $2^{\varepsilon_A+\varepsilon_B}f_Af_B$. Accordingly, the mantissa LSB+2 bit in the 9-bit log-adder is replaced by the proposed C-Adder, which injects an upward correction at that fractional weight to statistically compensate for the negative bias and reduce the mean/typical error.

In the proposed CIM, each 8-bit input and weight is converted into a logarithmic representation composed of a 3-bit exponent and 6-bit mantissa, as illustrated in Fig. 3. The position of the most significant '1' in the bit sequence of W is

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 20,2026 at 12:12:55 UTC from IEEE Xplore. Restrictions apply.



(b) Input			Output	
A	B	C_{in}	S	C_{out}
0	0	0	1	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	1
1	0	0	0	1
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

Fig. 4 Proposed C-adder: (a) schematic, (b) truth table, and (c) 9-bit configuration.

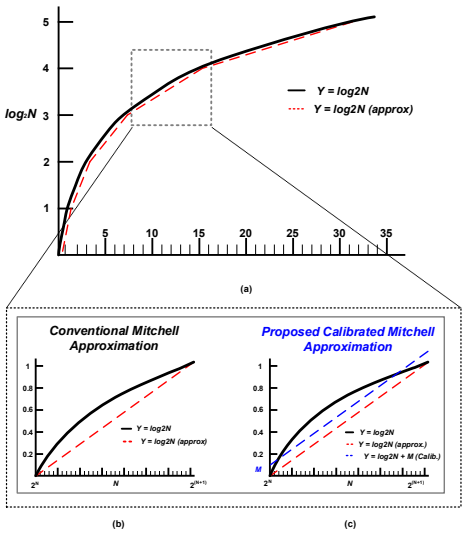


Fig. 5 Logarithmic approximation results. (a) ideal $\log_2 N$ curve vs. Mitchell. (b) error region of conventional Mitchell approximation. (c) Calibrated approximation with C-adder.

detected, and its exponent value is encoded into 3 bits and stored in L [8:6]. For example, if the most significant '1' of W is located at the 2^n position, the value n is encoded in 3 bits. The remaining bits, excluding the leading '1,' are sequentially written into L [5:0] from the most significant bit. If the number of remaining bits is insufficient, zeros are padded to the lower bits. Through this process, a fixed-length logarithmic data format L = [k(3b), x(6b)] is generated.

The two log-domain operands L_W and L_I are summed by a single 9-bit adder to realize multiplication; the mantissa carry naturally ripples into the integer field, as summarized in equation (4).

$$\underbrace{k_W}_{3b} + \underbrace{k_I}_{3b} \xrightarrow{+mantissa\ carry} \underbrace{K}_{4b}, \qquad \underbrace{x_W}_{6b} + \underbrace{x_I}_{6b} = \underbrace{X}_{6b} \tag{4}$$

A calibration adder is inserted in this 9-bit path to bias the result positively and mitigate the underestimation of MAM. The final output is a 10-bit normalized log word, [K(4b) : X(6b)].

The logarithmic-domain result L_{out} = {K[3:0], X[5:0]} is restored into 16-bit binary data. In this work, a barrel shifter–based reconversion structure is employed to perform this operation in a single clock cycle. A logic '1' is written to the MSB L[15] of a 16-bit register, and the following bits L[14:9] are sequentially filled with the mantissa bits X[5:0].

$$R \leftarrow \underbrace{1}_{bit15}\ \underbrace{X[5:0]}_{bit14..9}\ \underbrace{0\cdots0}_{bit8..0} \tag{5}$$

The upper 4-bit exponent K[3:0] is interpreted as the shift amount, and the register R is right-shifted by Kb. Each bit of K [3:0] corresponds to a shift condition of 1, 2, 4, and 8 positions, respectively. These four stages are cascaded, enabling a maximum 15-bit shift to be executed within a single clock cycle. Consequently, a scaling factor of 2^k is applied, and the final approximate multiplication result is restored in the binary domain.

This Mitchell approximate multiplication approach eliminates complex iterative additions or accumulations, enabling multiplication to be completed within a single clock cycle. As a result, the multiplication path is simplified into three stages: logarithmic conversion, single addition, barrel-shifter–based reconversion, significantly reducing both computation latency and power consumption compared to a conventional 8-bit multiplier (requiring up to seven additions) or a Booth multiplier (requiring about three accumulation cycles).

C. C-Adder : Calibration Adder

The Mitchell approximation performs a linear approximation in the logarithmic domain, which consistently underestimates the true value, causing a negative bias. To address this, a C-adder is introduced. As shown in Fig. 4(a), the C-adder maintains a full-adder-compatible transistor structure with one additional logic gate that enforces a +1 bias under most input conditions. This bias is applied except when all three inputs (A, B, C_{in}) are logic '1', where the circuit behaves identically to a conventional full adder. The corresponding truth table and the 9-bit adder chain, including the C-adder at the third least significant bit (LSB + 2) position, are shown in Figs. 4(b) and (c), respectively.

The effect of this calibration is illustrated in Fig. 5. The conventional Mitchell approach exhibits only negative deviation, while the proposed method introduces a controlled positive bias that compensates for underestimation and reduces total error magnitude. Some outputs become slightly overestimated, enabling partial error cancellation in global accumulation.

Implemented as a full-custom transistor-level design, the C-Adder occupies the same area as a standard full adder, providing accuracy improvement without area or delay overhead. Software-level evaluation of the calibrated logarithmic MAC operation (Fig. 6) confirms that the maximum and average errors are reduced from 5.0 % → 1.4 % and 3.7 % → 0.3 %, respectively.

D. Computing Element

To overcome the limited parallelism of conventional digital CIM, the proposed architecture employs a CE–based parallel processing scheme. The entire 64 × 400 array is partitioned into 8 × 16 sub-arrays, each equipped with an SCB responsible for local multiplication. Although this

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 20,2026 at 12:12:55 UTC from IEEE Xplore. Restrictions apply.

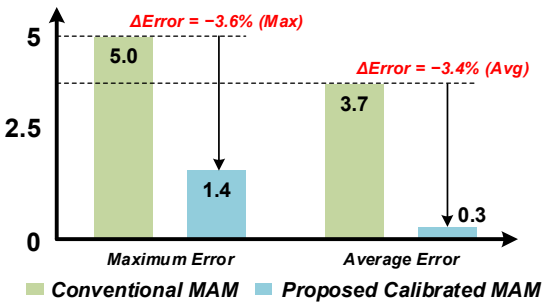


Fig. 6 MAC Error Comparison.

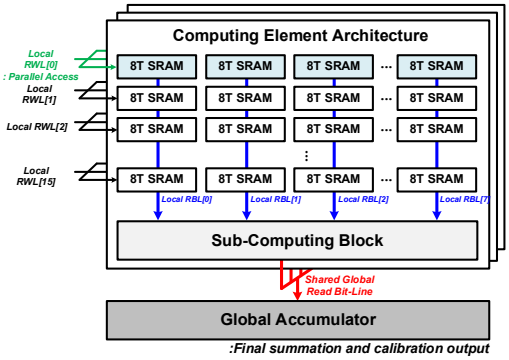


Fig. 7 CE-based parallel architecture with local and global RBL hierarchy.

hierarchical division slightly increases the area due to additional SCBs, it enables parallel operation among CEs, thereby reducing overall latency.

Each CE features an independent local RBL, allowing different CEs in the same column to activate separate rows simultaneously without interference. In particular, same-index rows across sub-arrays can be activated and processed in parallel under the top-level controller schedule, improving effective throughput. The partial products generated within each CE are collected through the global RBL, where they are accumulated by the global accumulator to produce the final output. This local–global RBL separation (Fig. 7) provides a deterministic one-to-one mapping to the global accumulator (one partial product per CE per MAC step), ensuring that parallel CE outputs are accumulated without omission or double counting and improving throughput for CNN workloads.

III. Implementation Result

The proposed CIM architecture was fabricated using a 28-nm CMOS technology, and the chip layout is presented in Fig. 8. The circuit operates at a 1 V supply voltage with a system clock of 100MHz. The macro performs 8-bit input and weight MAC operations, producing a 20-bit output. The total circuit area, including SRAM, occupies 0.246 mm².

To identify where the energy efficiency is improved, the proposed Mitchell-based multiplication block was compared with a cyclic-adder baseline and an FA×8 baseline. The proposed block completes an 8-bit operation in 1 cycle (vs. 8 cycles for the cyclic-adder baseline) and consumes ~486× lower average power due to avoiding TGFF-dominant switching; compared to the FA×8 baseline at the same 1-cycle latency, it reduces average power by ~60.8% and area by ~58.6%.

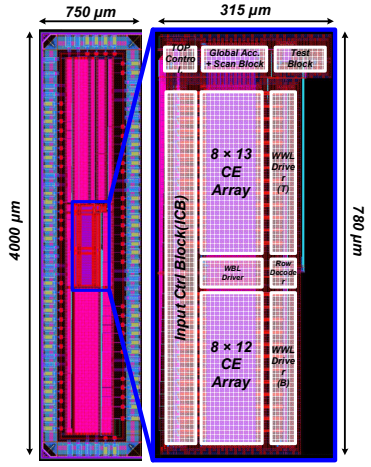


Fig. 8 Chip layout.

TABLE I. Comparison with Prior Works and Summary

	JSSC'24 [9]		ISSCC'22[10]	VLSI'24 [11]		This Work
Process	28nm		28nm	28nm		28nm
Supply Voltage [V]	0.65-0.9		0.6-1.0	0.5-0.9		1
Density [kb]	1000		96	16		25.6
Input Precision	4	8	8/16 (INT) 16/32 (BF/FP)	FP8, BF16, INT8		8
Weight Precision	4	8	8/16 (INT) 16/32 (BF/FP)	FP8, BF16, INT8		8
Energy Efficiency [TOPs/W]	89.45-115.6	22.02-28.55	36.5 (INT 8) 29.2 (BF 16) 3.7 (FP 32)	68.4 (INT8) 161.8 (FP8) 65.2 (BF16)		38.65
Network model	ResNet-20		ResNet-50	VGG-16	ResNet-20	ResNet-20
Dataset	CIFAR-10		ImageNet	CIFAR-10		CIFAR-10
Software Accuracy	92.04	91.97	75.87	92.97	91.71	91.86

A detailed comparison of the proposed design with prior works is summarized in Table I, where the proposed architecture achieves an area efficiency of 0.05003 TOPS/mm² and energy efficiency of 38.65 TOPS/W. In software-level evaluation using the ResNet-20 model on the CIFAR-10 dataset, the proposed design achieved an inference accuracy of 91.86%, which is higher than the 91.27% accuracy obtained without calibration. This improvement was achieved without any additional area or power overhead.

IV. Conclusion

In this work, an approximate multiplication-based CIM architecture with calibration was proposed to enhance the accuracy and efficiency of digital in-memory computing. By introducing a c-adder into the Mitchell-based logarithmic multiplication, the proposed design effectively compensates for the inherent underestimation error without additional area or power overhead. The CE-based architecture, featuring local–global RBL separation, enables parallel processing among multiple CEs and alleviates the row-by-row latency bottleneck of conventional digital CIMs. As a result, the proposed D-CIM achieves balanced improvements in energy efficiency, computation accuracy, and structural scalability, demonstrating its potential for efficient AI and edge-computing applications.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 30,2026 at 12:12:55 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] W. A. P. Maass et al., "A 28-nm Compute SRAM With Bit-Serial Logic Arithmetic Operations for Programmable In-Memory Vector Computing", *IEEE J. Solid-State Circuits (JSSC)*, vol. 55, no. 1, pp. 99–111, Jan. 2020.

[2] Xin Si, et al., " A 28-nm 16-kb Sign-Extension-Less Digital-Compute-in-Memory Macro With Extension-Friendly Compute Units and Accuracy-Adjustable Adder-Tree", *IEEE Trans. Very Large Scale Integration (VLSI) System*, vol. 30, no. 11, pp. 2164-2168, Sep. 2024.

[3] W. Jia and Y. He et al., "Pro-Cache-CIM: A 28 nm 69.4 TOPS/W Product-Cache-based Digital-Compute-in-Memory Macro Leveraging Data Locality Pattern in Vision AI Tasks", in *Proc. IEEE Custom Integrated Circuits Conference (CICC),* Apr. 2025.

[4] J. Cho and W. Jung, "Hardware-Software Co-Design for Analog Compute-in-Memory Accelerators Using 1-bit Sense Amplifiers," *IDEC Journal of Integrated Circuits and Systems (JICAS)*, vol. 11, no. 1, pp. 44–48, 2025.

[5] J. Wang, et al., "A 28-nm Compute SRAM With Bit-Serial Logic/Arithmetic Operations for Programmable In-Memory Vector Computing,", *IEEE Journal of Solid-State Circuits(JSSC)*, vol. 55, no. 1, pp. 76–86, Jan. 2020.

[6] Y. He, et al., "34.7 A 28nm 2.4Mb/mm^2 6.9-16.3TOPS/mm^2 eDRAM-LUT-Based Digital-Computing-in-Memory Macro with In-Memory Encoding and Refreshing", *IEEE Int. Solid-State Circuits Conf. (ISSCC), pp. 578-580, Feb. 2024*.

[7] J. N. Mitchell, "Computer Multiplication and Division Using Binary Logarithms", *IRE Transactions on Electronic Computers*, vol. EC-11, no. 4, pp. 512–517, Aug. 1962.

[8] H. Yoon, D. Kim, G. Lee, and S. Cho, "A DRAM-Based Process-in-Memory Using Data Redundancy and Differential Bit-Line Computation," *IDEC Journal of Integrated Circuits and Systems (JICAS)*, vol. 10, no. 1, pp. 5–10, 2024.

[9] P. Wu, et al., "An 8b-Precision 6T SRAM Computing-in-Memory Macro Using Time-Domain Incremental Accumulation for AI Edge Chips", *IEEE Journal of Solid-State Circuits(JSSC)*, vol. 59, no. 1, pp. 2297–2309, July. 2024.

[10] P. Wu, et al., "A 28nm 29.2 TFLOPS/W BF16 and 36.5TOPS/W INT8 Reconfigurable Digital CIM Processor with Unified FP/INT Pipeline and Bitwise In-Memory Booth Multiplication for Cloud Deep Learning Acceleration", *IEEE Int. Solid-State Circuits Conf. (ISSCC)*, pp. 254–256, Mar. 2022.

[11] R. Guo, et al., "A 28nm 4170-TLOPS/W/b and 195-TFLOPS/mm^2/b Multiply-Free Fully-Digital Floating-Point Compute-In-Memory Macro with Mitchell's Approximation", 2024 *IEEE Symposium on VLSI Technology and Circuits(VLSI Technology and Circuits)*, pp. 1–2, 2024.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:12:55 UTC from IEEE Xplore. Restrictions apply.